

# Zero-Knowledge Identity Verification

Pranav Kumar  
Department of Computer Science and  
Engineering  
Ajay Kumar Garg Engineering College  
Ghaziabad, India  
baliyanpranav5@gmail.com

Nikita Gupta  
Department of Computer Science and  
Engineering  
Ajay Kumar Garg Engineering College  
Ghaziabad, India  
nikitagupta7668@gmail.com

Param Srivastava  
Department of Computer Science  
and Engineering  
Ajay Kumar Garg Engineering College  
Ghaziabad, India  
paramsrivastava5@gmail.com

Parth Singh  
Department of Computer Science and  
Engineering  
Ajay Kumar Garg Engineering College  
Ghaziabad, India parths.singh2003@gmail.com

Vivek Agarwal  
Department of Computer Science  
and Engineering  
Ajay Kumar Garg Engineering College  
Ghaziabad, India vivekagarwal292@gmail.com

**Abstract**—Everybody is these days plunging into decentralized applications, blockchain, and digital identity. And honestly, it's a rendering that the ancient method of authenticating yourself looks nice, outdated and risky. Whenever you give up your personal info, you are just wishing it does not find its way into a data leak or get misused. Precisely, this is the reason that we constructed a new path to manage identity checks that really care about your privacy. This is what we are doing: **our framework is based on Ethereum Attestation Service (EAS) and Zero-Knowledge Proofs (ZKPs). To begin with, we generate off-chain attestations based on the EIP-712 standard. Your signature remains verifiable; however, your confidential information doesn't go anywhere and through which we squeeze these attestations. SP1 using zero-knowledge virtual machine (zkVM) this step checks everything twice, the construction, the encryption, the logic, all of it. When it's all good then the system will spit a short, non-interactive Groth16 or Plonk (if)SNARK proofs known as zero-knowledge proofs( you're curious). This evidence makes things private and at the same time accurate. There is the interesting side of it: you can check it immediately in your local devices using a super lightweight browser or with a Node.js app WebAssembly verifier. It does not require any middlemen and there is no need for extra trust. You can send evidences, in case you wish to using Solidity contract on-chain. This allows the system to issue new fraudulent statements such as isOver18 = true without displaying your actual age or any other personal data. So what does this mean? You earn greater confidence, enhanced interoperability and connections through decentralize systems. Transparency is what you have with this of ZKP attestation and actual privacy. It's actually practical, as well, can be used with KYC, DeFi, age-requiring app checks, and secured access controls. Essentially, it is a huge leap higher on behalf of anonymity and trust in electronic self.**

**Keywords**—Ethereum Attestation Service (EAS), Zero-Knowledge Proofs (ZKPs), zkVM (SP1), EIP-712, Verifiable Attestations, Groth16, Plonk, WebAssembly (WASM) Verifier, Solidity Verifier Contract, Privacy-Preserving Identity Verification, On-Chain Proof Verifications, Decentralized Identity, Blockchain Security, Trustless Verification, Web3.

## I. INTRODUCTION

As more individuals use decentralized apps, digital Web3 identity systems, the necessity of safe and secure, and finance identity verification that is privacy-oriented is more in need than ever before. The need of apps, digital money, and Web3 identity systems never has secure, privacy-first identity verification how it has been now. Earlier it was used to be like you give out your own details- government id, biometrics, addresses to some central service provider or authority, and pray they do not lose it. Sure, that gets you in the door, but it opens up a can of worms: identity theft, loss of privacy even surveillance and when you begin to place on that sort of sensitive data you use decentralize network, then you are just throwing out the entirely the concept of user-controllability and trustless systems which blockchains represent supposed to stand for.

It becomes even more provocative with such items as DeFi, age checks, online. voting, or when it is required to obey KYC and AML regulations. You must have to show what you are, yet you can not run the chance of showing what you are private details to the world. Ethereum Attestation Service (EAS) is where the latter comes in, EAS allows you to make and check digital attestations on the blockchain essentially, signed and immutable assertions concerning a user or asset. They are malleable, and even issuing them off-chain, not so as to tie up the net, but to prove things on-chain when you need to. It sounds like the answer, but there is a condition: EAS frequently implies that the verifier will be able to see all the details in the attestation, Sometimes, that's too much. Take age verification - the attestation may testify that someone is over 18, but it might also tell the very date they were born. That's private info, and it ought not to be included in the deal. So, we need something more an additional covering which hold what keep that which is private yet allows people to demonstrate what counts.

Zero-knowledge proofs (ZKs) allow you to do things nicely, unique: one individual may demonstrate to another that a statement is true, but as without relaying any additional details, That's the magic. At this point, however, combining ZKPs with the Ethereum Attestation Service (EAS), you have attestations not merely verifiable, but private, too. This is the way it works: you create a structured off-chain attestation and sign it with EIP-712. That standard store electronic signatures in a safe and verifiable manner. After getting the attestation, a Zero-Knowledge Virtual Machine (zkVM) at this point, it comes to be SP1. This thing checks the form of the attestation, ascertains the signature, and secures any other regulations, which are attached to the claim, actually stand. All of this occurs behind the curtain, thus no secret data is revealed out. Once it is finished, the zkVM spews out a non-interactive, short one zero-knowledge proof (A zkSNARK), based on sophisticated schemas like Groth16 or Plonk. This evidence is a privacy first certificate equivalent of a tiny size everything checks out.

You do not have the appearance of old-school attestations that need not have much weight of equipment to prove it, Even web browsers or by Node.js applications with the use of WebAssembly (WASM) verifier. It implies that there is no necessity of everyone depending on a central authority. The entire system is open, and can continue to develop. If you want more trust, you may put these evidences into an on-chain Solidity verifier contract. Then, their validity can be checked by any person on the blockchain, and history is everlasting. Once the on-chain check is complete, it is possible to issue new attestations, making sure that an individual is old enough or that he or she has passed a KYC check, no sensitive details being revealed at all. So in this hybrid, where the evidences are constructed off-chain and checked on-chain, provides you with scale, transparency and authentic privacy.

The combination of the EAS and the zero-knowledge proofs is a genuine thing that modifies the game to the decentralized identity. Suddenly, you've had privacy, verifiability, and the freedom to make extensions on it all at once. It is quite an improvement over centralized identity services, which concentrated a great deal of power (and danger) in one hand. Here, trust gets cryptographically disseminated, and this is compatible with the entire spirit of decentralization and because you can establish theories without sharing personal information, this strategy is effective with tons of things, financial compliance, decentralized governance, safe credentials, whatever. By mixing open notions of secrecy, a solid you have attestations with the secrecy of zero-knowledge. lay groundwork to privacy-first identity in Web3.

## II. LITERATURE SURVEY

The concept of identity management processes was initially based on the centralized organizations which resulted in the issue of surveillance, information leakage and unprotected transfer of personal information [1]. The Self-Sovereign Identity (SSI) architectures are aimed at increasing the level of user

control, yet they are limited by the problem of decentralization, interoperability, and scalability, even though they increase the level of user control. The challenges led to identity schemes based on blockchain and offer feasible verification and reduced reliance on centralized authorities. The cryptographic primitives form the basis of the systems to demonstrate the user attributes without disclosing the raw data but disclosing the metadata and creating storage overheads to the public chains [2]. EAS enhances decentralized identity that is concerned about structured attestations on Ethereum, EIP-712 gives the explanation of the hash of structured data and you need not worry about replay attacks. Moreover, it also stores signatures and renders them reliable even when you are not online. Nevertheless, in the case where the attestations or metadata is on-chain, the relationships based on identity can be disclosed even in the case where the identity is not directly related to the relationship being measured by the metadata or the attestation itself. The article by [3] states that Zero-Knowledge Proofs (ZKPs) are a new privacy-saving identity verification method, and users do not need to provide sensitive information to the server in order to attest to the properties. A number of proof systems including zk-SNARKs, Bulletproofs and STARKs provide a range of efficiency versus trust assumptions trade-offs. The private KYC, anonymous credentialing and selective disclosure have been implemented using identity protocols on ZKP. The filecoin SNARKs indicate that large-scale ZK systems can exist.

## III. RESEARCH GAPS AND LIMITATIONS IN EXISTING SOLUTIONS

System that uses EAS-based verification requires verifiers to access full attestation data, which in turn leads to over exposure of sensitive data. The drawback that current system holds is that they lack a cryptographic layer that discloses the data selectively without compromising integrity of verification. Mechanisms that can only share proof of compliance without over exposing the details are limited. Sensitive metadata such as wallet addresses may be revealed by public attestations. Without the use of ZKPs, on-chain attestations can exposed user attributes and violate privacy-by-default principles. Relational privacy or verifier proof interactions are not natively protected by decentralized identity solutions. High quality hardware is required to for heavy ZK proof generation. Optimized libraries are missing in software that can verify ZKK proofs faster. Centralized ZK proof breaks decentralization guaranteed by introducing trust assumptions. Generation and verification of zk-SNARK impose very high computational load on constrained devices. Latency may be introduced due to this. Frequent on-chain identity checks are not economically feasible. There are no unified standards that provides with cross-chain identity portability and credential reuse.

Incompatibility in protocols between other block chain platforms. There are no platform for universal identity verification or for verifying ZK credentials. Single point cryptographic risk is created as various schemes like Groth16 heavily rely on

trusted setups. Long term security gaps are an issue as many identity protocols do not address post quantum resistance. System that provides us with transparency are not fully utilized due to performance tradeoffs and tooling maturity. Frameworks today operate either on complete on-chain or off-chain without hybrid trust models. Infrastructure for off-chain proof computation with on-chain verification Assurance is limited. There is no standard mechanism to get light-weight client-side credentials from attested identity proofs. On many occasions user has to rely on centralized issuers for recurring verification and to reduce autonomy. Limited tool availability for proof generation and private credential storage. There is still dependence on centralized authorities for identity revocation and validity updates. Ready-made components support for common identity checks is missing in present libraries (e.g., age, KYC). Manual intervention in circuit development may lead to circuit design flaws and multiple bugs. There are no standardized methods for proving credential revocation without revealing user identity. It is difficult to synchronize off-chain proof state with that of on-chain trust state. Timestamp is disclosed due to limited cryptographic primitives for freshness of proofs. Advanced knowledge of cryptography and circuit design is required and adoption is limited. Debugging is very complex as the domain remain under-developed. There are limited SDKs which provide seamless integration of ZK, wallet interaction layers and attestations.

#### IV. PROPOSED MODEL

The suggested identity verification system, illustrated in Fig .1, is designed in such a way that it is simple to use and implement. All components of the framework, including offchain attestations and zK proof checks in client software are based on well-known cryptographic and blockchain standards to ensure interoperability. The attestations are all formatted in a manner of the EIP-712 format that has clear instructions on how the typed data should be signed and verified. This guarantees the signature stays interoperability between the wallets, tools and decentralize apps. The users and developers do not have to modify any existing specifications or cryptographic mechanisms, as the system deliberately adheres to well established standards to ensure the protection of verifications integrity and composability to everyone. Before attestations can be consumed within the zkVM, they had to be formatted, signed and stored in an open and standardized format.

As with any cryptographic or blockchain framework, the system introduces a number of technical terms and acronyms. For clarity and accessibility, all acronyms such as EAS (Ethereum Attestation Service), ZKP (Zero-Knowledge Proof), zkVM (Zero-Knowledge Virtual Machine), WASM (WebAssembly), and SNARK (Succinct Non-Interactive Argument of Knowledge) should be defined at first use. Once introduced, they may be used throughout the system documentation without ambiguity. However, the use of the abbreviation in user-facing attestations or in front-end applications should be minimize to avoid any confusion and ensure that proofs remain human-readable when checked.

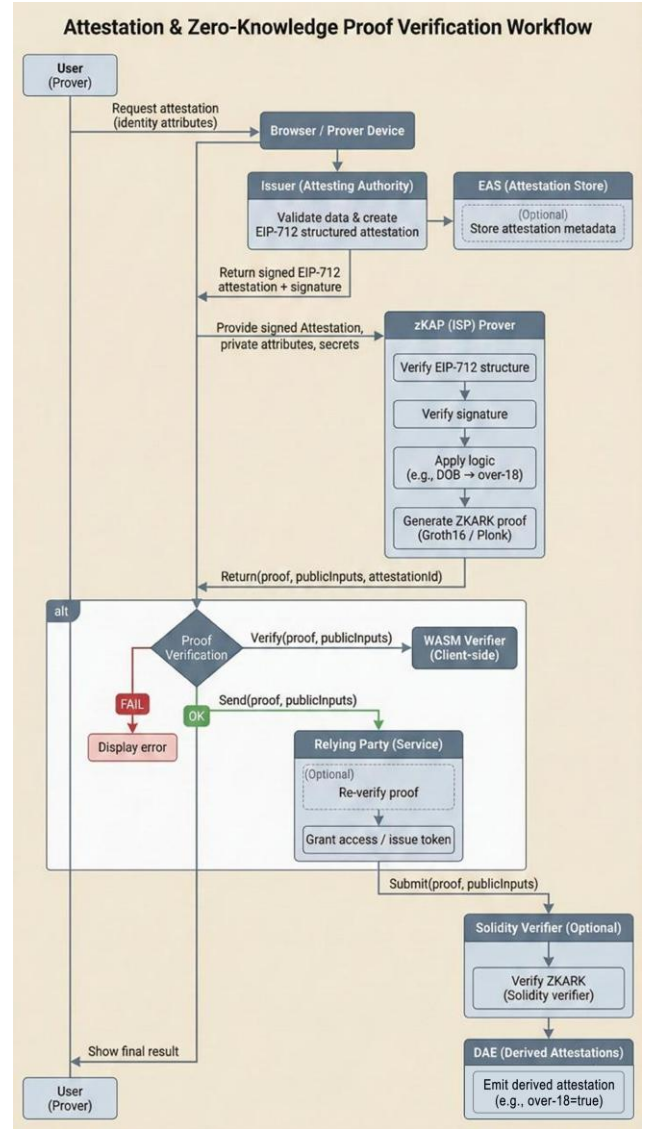


Fig. 1: Attestation & Zero-Knowledge Proof Verification Workflow.

To maintain precision, attestations uses standardize data types and formats. For example, dates should be written as ISO 8601 format (YYYY-MM-DD) rather than application specific encodings, and numerical fields should always be normalized to prevent ambiguity across verifiers. Digital signatures are represented as hexadecimal encoded strings, and proof objects generated by zkSNARK frameworks are serialized into JSON-compatible structures for interoperability. Care should be taken to avoid mixing data representations (e.g., decimal vs. hexadecimal) when handling attestations and proofs, as this could lead to verification mismatches. Whenever possible, the zero-padding and canonical encoding must be applied to maintain consistency.

Proof generation is performed in the zkVM, and the struc-

ture of attestation, a digital signature, and logical conditions are verified. Proofs are intended to be small and low-level, allowing them to be verified quickly in web browsers or in a Node.js application using a WASM verifier [4]. As in:

#### A. Formal Model

**1. Attestation Structure.** Following EAS and EIP-712, we define the attestation  $A$  as:

$$A = \{uid, S, I, R, T_{exp}, D\}$$

where  $uid$  is the attestation identifier,  $S$  the schema ID,  $I$  the Issuer address,  $R$  the Recipient,  $T_{exp}$  the expiration time, and  $D$  the encoded attribute payload (e.g., DOB).

**2. EIP-712 Signature.** The EIP-712 digest  $h$  is computed as:

$$h = \text{Keccak256}("\x19\x01" \parallel \text{domSep} \parallel \text{hashStruct}(A))$$

The Issuer signs the digest using its private key  $sk_I$ :

$$\sigma = \text{Sign}(sk_I, h)$$

**3. Zero-Knowledge Circuit.** Let  $C$  denote the zk-SNARK relation with public inputs  $x$  and private witness  $w$ :

$$R_C(x, w) = 1 \iff \begin{cases} \square \square \text{VerifySig}(pk_I, h(A), \sigma) = 1 \\ \square A.S = S_{\text{expected}} \\ \square \Phi(D) = 1 \end{cases}$$

Public inputs:  $(pk_I, S_{\text{expected}}, t_{\text{now}})$ . Private inputs:  $(A, \sigma, D)$ .

**Predicate Logic (Age Example).**

$$\Phi(D) = \begin{cases} 1 & \text{if } (t_{\text{now}} - D_{\text{dob}}) \geq 18 \cdot 31536000 \\ 0 & \text{otherwise} \end{cases}$$

**4. Proof System.** Using a zk-SNARK system (e.g., Groth16/Plonk), the prover generates:

$$\pi \leftarrow \text{Prove}(PK, x, w)$$

and the verifier checks:

$$b = \text{Verify}(VK, x, \pi) \in \{\text{True}, \text{False}\}$$

**5. Privacy.** The proof reveals no private data:

$$L(\pi) = \emptyset$$

ensuring zero-knowledge: only the truth of  $\Phi(D)$  (e.g., “over 18”) is learned.

where  $f$  represents the zkSNARK proving function operating over structured attestations. Proof numbering and identifiers should be managed automatically by the framework rather than manually by developers to avoid discrepancies.

#### B. Algorithm

The core logic of the privacy-preserving identity system is encapsulated in the following protocol, which ensures that the prover can only generate a valid proof  $\pi$  if they possess a valid signature  $\sigma$  from the authorized issuer  $I$  on the correct schema  $S$ .

---

#### Algorithm 1 End-to-End Privacy-Preserving Verification Protocol

---

**Require:** Identity Attributes  $D$  (e.g., DOB), Issuer Private Key  $sk_I$ , Proving Key  $PK$ , Verification Key  $VK$ , Current Time  $t_{\text{now}}$

**Ensure:** Verification Result  $b \in \{\text{true}, \text{false}\}$

```

1: Phase 1: Off-Chain Attestation (Issuer)
2:  $S \leftarrow \text{SchemaID}(\text{"AgeVerification"})$ 
3:  $A \leftarrow \{uid, S, I, R, T_{exp}, D\}$ 
4: {Hash structured data per EIP-712 spec}
5:  $h \leftarrow \text{Keccak256}("\x19\x01" \parallel \text{domSep} \parallel \text{hashStruct}(A))$ 
6:  $\sigma \leftarrow \text{Sign}(sk_I, h)$ 
7: return Signed Attestation  $(A, \sigma)$  to User
8: Phase 2: Proof Generation (User / zkVM)
9: Public Inputs  $x \leftarrow \{pk_I, S, t_{\text{now}}\}$ 
10: Private Witness  $w \leftarrow \{A, \sigma, D\}$ 
11: Initialize zkVM with Circuit  $C$ 
12: if  $\text{VerifySig}(pk_I, h, \sigma) = \text{true}$  then
13:   abort (Invalid Signature: Data Tampered)
14: end if
15: if  $\text{CheckSchema}(A, S) = \text{true}$  then
16:   abort (Invalid Schema: Mismatched format)
17: end if
18: Execute Predicate Logic (e.g., Age Check):
19: if  $(t_{\text{now}} - D_{\text{dob}}) < (18 \times 31536000)$  then
20:   abort (Condition Failed: User Underaged)
21: end if
22: return  $\text{Prove}(PK, x, w)$ 
23:
24: Phase 3: Verification (Verifier)
25: {Verifier sees only  $\pi$  and  $x$ , never  $D$ }
26:  $b \leftarrow \text{Verify}(VK, x, \pi)$ 
27: if  $b$  is true then
28:   Grant Access / Issue Derived Token
29: else
30:   Deny Access
31: end if

```

---

#### C. Specific Advice

In this framework the zkVM function as the core in the system, so its correct implementation is essential for both efficiency and stability. It is too tiring to build everything from scratch each time, developers should design circuits in a modular and reusable way to keep projects scalable and easier to manage. Proof references are better generated dynamically rather than being hard coded, since this makes future upgrades simpler and reduces the risk of breaking compatibility. When working with zkSNARK frameworks like Groth16 or Plonk, it's important to handle parameter files carefully as reusing them in the wrong context can lead to invalid proofs. For a long period maintainability, circuit definitions and proving keys should remain separate from the main application logic, allowing updates without major code

rewrites.

#### D. Some Common Mistakes

Multiple recurring problems impact the overall usability and reliability of the framework for developers:

- Developers should never keep sensitive attestations or confidential attributes on-chain as this significantly weakens the overall privacy guarantees.
- issuers and verifiers using different schema structures can easily lead to broken proofs, which is why proper schema checking tools should always be used beforehand [5].
- Mishandling digital signatures, for example, using compressed instead of uncompressed public key signatures and vice versa, can cause verification problems.
- Proof validation in WASM environments should ensure that the same version of the cryptographic library is used to avoid unforeseen compatibility problems.

#### E. Performance Evaluation

The EAS-ZKP framework was looked at over three main stages: Presenting evidence, and Finally which we verified at scale Off the Blockchain as well as EIP-712 signature checks which require Very short processing time and typically complete within milliseconds. In these processes which take place off the chain their speed Is separate from how quickly the blockchain itself responds to requests.

As part of the framework, a zkVM set up with SPI functions as the proof generation execution environment. This virtual machine incurs a definable cost, reflecting the expense of performing the cryptographic operations in a secure manner. While proof generation remains the most resource-consuming activity, it however, remains suitably performant for everyday use cases in regard to age verification, eligibility verification, and identity attribute verification in decentralized applications.

The resulting zkSNARK proofs, produced using schemes like Groth16 or Plonk, remain compact and lightweight. Their small size allows fast transmission and efficient storage across different systems. Verification is also highly optimized: WASM-based verifiers running in browsers or Node.js confirm proofs within milliseconds.

### V. DISCUSSION AND CONCLUSION

Privacy, decentralization, trustlessness, and verifiability Continue to be tough goals for modern identity verification Systems which in practice work together. The combination of zero-knowledge techniques with decentralized attestation Models provides a robust direction for developing identity frame Works that preserve user privacy at the same time which also do not trade off privacy for transparency of the data auditing Such a design does away with central authorities. Identity proofs which are kept cryptographically Open for feedback and very much on the user experience. Conventional attests which put forth private data, but zero knowledge methods allow verification of state Presentations of data without going into the details of the underlying information. When EAS is a part of the zkVM execution, we have that EAS forms

a trusted environment where EIP-712 attestations validated without depending on intermediaries. This process is one which confirms authenticity and structure as well as which puts a stop to unnecessary disclosure during verification. We have put forth an architecture which increases user ownership by way of support for self managed verifiable credentials and reduction of the constant issue of which issuers are in control. Individuals may present credentials across many platforms which include browsers, mobile apps, or decentralized apps through light weight WebAssembly based proof checks. We see in this how privacy, transparency and independence come together in the Web3 model which does not force trade off between security and ease of use. Also a great asset of this model is its flexibility across industries. What we put forth is a model in which zero knowledge proven credentials may be used in different ecosystems and networks which in turn supports inter operability and long term portability.

Table 1 shows the main points about the design and checking process. It compares our system with other identity systems. Most other systems use one step to check identity. They share user data and depend a lot on blockchain work. Our system does not share user data. It only shares proof results in three steps. We make proofs safely inside the zkVM system. We use zero-knowledge proof methods. This way, we use the blockchain less. It also keeps user data private and makes the system more trustworthy.

Table I  
Comparison Between Existing Approaches and Proposed System

| Metric              | Existing Approaches                         | Proposed System                                           |
|---------------------|---------------------------------------------|-----------------------------------------------------------|
| System Components   | Blockchain with ZKP support                 | EAS, EIP-712, zkVM (SPI), zkSNARK, WASM, Solidity         |
| Verification Flow   | Mostly single-stage verification            | Three stages: attestation, proof generation, verification |
| User Data Exposure  | Attestation data or meta-data often visible | No raw attributes shared, only proof results              |
| Proof Generation    | External circuits or server-based systems   | Inside zkVM with signature and logic checks               |
| Proof Techniques    | Usually one ZK method                       | Supports Groth16 and Plonk                                |
| Verification Method | Primarily on-chain                          | Browser/Node verification with optional smart contract    |
| Blockchain Usage    | Frequent on-chain operations                | On-chain verification only when required                  |
| Trust Dependency    | Relies on issuer or verifier services       | No mandatory intermediary                                 |
| Architecture Type   | Fully on-chain or fully off-chain           | Hybrid off-chain proof with optional on-chain validation  |

We work together on a new system for digital identity. It uses zero-knowledge cryptography. It runs on zkVM. It follows clear rules for proof. We plan to add features later. These include post-quantum proofs, combining proofs, lists to

stop bad identities, and working across different blockchains. This system lets you control your digital identity. It keeps your data private. It works with many systems. It can be checked anywhere. We want privacy to be key in future finance and governance.

We use standard proofs and zero-knowledge checks. This helps stop identity systems from being different and broken. People who check IDs do not need to trust the people who gave the ID. This makes it easier to use in many apps. The system can grow bigger and still keep your data private. It also shares less data and has fewer control points. This stops spying and wrong use of data. We make identity systems safer and more reliable.

We built a system that works well in many real life tasks that need trusted digital identity. In decentralized finance, you can prove you follow rules without showing private details. This lets you use financial services safely and keep your privacy. For sites that limit age or share media, you can prove you meet age rules without giving your birthday or ID. In voting and government systems, you can prove you have the right to vote while staying anonymous and safe from pressure. In health and education, you can prove your qualifications or coverage without showing private information. This shows the system works in many areas.

## REFERENCES

- [1] Duc Anh Luong and Jong Hwan Park, Preprint on ResearchGate, 2021–2023.
- [2] Sai Vamsi Kiran Gummadi, “Zero-Knowledge Proof-Based Identity Verification in DeFi,” arXiv Preprint, 2023.
- [3] Enis Karaarslan and Beste Akdik “Toward Reliable Disaster Data Sharing With Blockchain, Decentralized Identity, and ZKPs,”
- [4] Filecoin Project, “Zero-Knowledge and the Filecoin Network,” Filecoin Blog, Jul. 2021.
- [5] S. Liu, “Privacy-Preserving Age Verification based on Improved Techniques,” in *CONPRO: IEEE Workshop Proceedings*, 2025.
- [6] S. Vijayakumaran, “Privacy-Preserving Aadhaar-based Age Proofs,” IIT Bombay technical report, 2024.
- [7] Ryan Lavin, “A Survey on the Applications of Zero-Knowledge Proofs,” arXiv:2408.00243, Aug. 2024.
- [8] Succinct Labs / SP1, “SP1 zkVM: Zero-Knowledge Virtual Machine,” documentation/spec, 2023–2024.
- [9] EIP-712 Working Group, “Ethereum EIP-712: Typed Structured Data Hashing and Signing (spec / suites),” 2021–2022.
- [10] Ethereum Attestation Service (EAS), “EAS Documentation and SDK,” 2023–2024.
- [11] Xonoxitron, “ZK-WASM: Zero-Knowledge Proofs compiled to WASM,” GitHub Repository, 2023.
- [12] N. Wang et al., “Blockchain-Based Privacy-Preserving Federated Learning (BPFL),” *Journal / Conference Article*, 2024.
- [13] L. Zhou, “Leveraging Zero Knowledge Proofs for Blockchain-Based Identity Management,” *Computers & Security*, 2024.
- [14] Survey: “Zero-Knowledge Proof Frameworks and Implementations,” arXiv Survey, 2024–2025.
- [15] zkSecurity Blog, “Ten ZK Papers that shaped modern ZK,” 2024.
- [16] Project Documentaries and Repositories “ZK-WASM and client-side verification approaches,” , 2022–2024.
- [17] Survey Preprint, “Zero-Knowledge Proofs in 5G and Beyond Technologies,” , 2025.
- [18] Zhangquan Wang et al “Lightweight Zero-Knowledge Authentication Scheme for IoT-Embedded Devices (LZIA),” arXiv Preprint, 2024.
- [19] N. Sheybani, “Zero-Knowledge Proof Frameworks: A Systematic Survey,” arXiv:2502.07063, Feb. 2025.
- [20] Protocol Labs, “The Future of ZK Proofs,” Research Report, Nov. 2023. (conference / workshop Paper), 2025.